

PRODUCT + ENGINEERING HANDOFF

Roarball

Small shots.
Big roars.

Voice-powered 3D mini-golf.
Godot + typed GDScript.
Kotlin only when justified.

12 authored holes

2 worlds

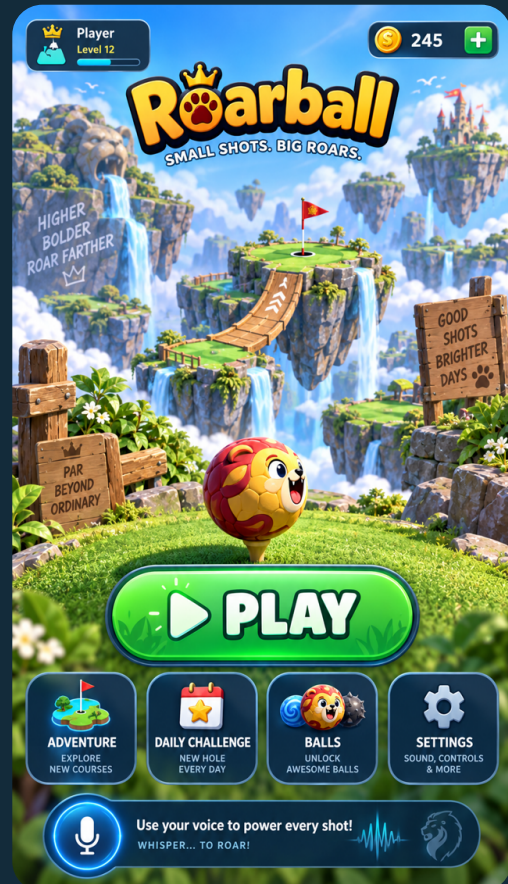
3 cosmetics

Version 1.0 · September 5, 2026

Android-first implementation plan

Planning package, not a runnable game.

Cover image: existing concept reference.



From concept to playable game

This handbook contains the full product and technical specifications. Editable Markdown, 60 issue bodies, schemas, examples, and an offline mockup gallery are included in the ZIP.

Scope rule: the written requirements override decorative or extra features shown in the concept images. There is no economy, mandatory screaming, cloud audio, or Flutter shell in the MVP.

- 01 Roarball — Product Requirements Document
- 02 Game Design and Rules
- 03 Technical Architecture
- 04 Voice Input, Calibration, and Privacy Specification
- 05 Physics, Obstacles, and Camera Specification
- 06 UI/UX Implementation Specification
- 07 Art Direction and Asset Pipeline
- 08 Content, Local Saves, and Data Validation
- 09 Android Delivery, Optional Kotlin, and Build Runbook
- 10 QA, Acceptance, and Release Evidence
- 11 Delivery Plan and GitHub Workflow
- 12 Decisions, Risks, and Open Questions
- 13 Sources and Verification Boundaries
- 14 Task overview and execution gates
- 15 Seven visual references with implementation notes

External platform/API facts refer to source IDs [S01–S15] in the Sources chapter. Gameplay thresholds and budgets are proposed decisions, not measured results. Full agent prompt: AI_AGENT_HANDOFF.md.

Roarball — Product Requirements Document

Version 1.0 · September 5, 2026 · Proposed implementation baseline

1. Product decision

Roarball is a mobile-first, single-player 3D mini-golf puzzle game. Players aim a cute lion-faced ball with touch and set shot power using a short, comfortable sound. A soft sound produces a gentle putt; a stronger calibrated sound produces a stronger shot. Touch-only power is an equal, permanent alternative.

Build the entire game and interface in **Godot 4.7.2 stable with typed GDScript**. The official archive listed this stable version when checked on September 5, 2026 [S01]. Use the integrated Jolt 3D physics engine, subject to a physical-device spike [S03]. Kotlin is a conditional Android integration language, not a second gameplay language. No Flutter shell, Unity bridge, or external physics extension is required by the proposed MVP.

The product promise is **“Small shots. Big roars.”** It describes personality, not a requirement to scream. Roarball is about precision, timing, and playful physical consequences. Loudness is not a score multiplier.

2. Goals and non-goals

Primary goals are to make the first shot understandable without a text-heavy tutorial, make quiet and strong shots both useful, deliver repeatable physical behavior, and translate the reference art into readable real-time mobile scenes.

The first playable milestone must prove one complete hole on an Android phone: enter, aim, choose power, shoot, roll, finish, save, retry. The visual slice must then prove one polished hole before all remaining content is produced.

MVP non-goals: multiplayer, online leaderboards, user accounts, cloud saves, advertising, in-app purchases, energy/hearts, paid retries, daily challenges, generated courses, AI services, speech recognition, voice uploads, realistic water simulation, destructible terrain, and an in-game level editor. The extra worlds and economy elements in the earlier mockups are future concepts, not committed scope.

3. Audience and platform assumptions

The design targets casual puzzle players and families who enjoy short, expressive play sessions. This is a proposed audience, not a validated market claim. Do not infer a child-directed regulatory classification from cute art alone; the publisher must make and review that decision before release.

Android phones are the first release target. Desktop editor builds support development and QA. iOS is a later delivery track; keep platform boundaries clean but do not claim an iOS release is included. Portrait is primary. A landscape fallback and tablet layout must remain usable; the gameplay does not depend on a narrow device-specific screenshot.

Sessions are intended to contain a few holes; hole duration is a design target of roughly 30–120 seconds, not a hard timer. There is no mandatory microphone use and no need to pronounce English words. UI copy starts in English with localization keys; Indonesian is a planned translation task rather than hardcoded mixed-language text.

4. Core player journey

Boot to Home without an immediate microphone permission dialog. “Play” resumes the next unlocked hole. A first-session choice offers Voice or Touch. Choosing Voice explains local sound-level processing, requests permission, checks room noise, and calibrates comfortable soft and strong sounds. Denial goes directly to the full Touch experience.

The player enters a safe introductory hole, drags to aim, holds the microphone button, produces a short sound, sees the exact power that will be committed, and releases to shoot. A drag into Cancel aborts without a stroke. In Touch mode, the same place contains a power slider and Shoot button.

The ball rolls under physics. Shot controls are disabled while moving. The player can inspect the course or pause. If the ball goes out of bounds, one penalty stroke is added and the ball returns to its last safe resting position. Finishing produces a results screen with strokes, par, earned stars, and progress. Next Hole and Retry are always clear.

5. Scope at release

Two worlds contain twelve authored holes: Cloud Cliffs (CC01–CC06) and Portal Peaks (PP01–PP06). Every hole has a playable `.tscn` scene, validated metadata, an authored par, a twelve-stroke attempt limit, and an internal reference solution. Metadata in this package is a design brief; it is not evidence that geometry has been built or solved.

Cloud Cliffs introduces putting, strength control, rebounds, ramps, moving gates, and combinations. Portal Peaks introduces paired portals, bounce pads, and later recombinations. Wind fans, lava environments, multiple moving platforms, and full physics trajectory prediction are deferred.

MVP collection has three cosmetics: Lion (default), Panda (unlock after CC06), and Robot (unlock after PP06). All share identical collider, mass, friction, and shot curve. No stat boosts, rarity economy, currencies, purchases, or random unlocks.

6. Functional requirements

ID	Requirement	Acceptance evidence
FR-01	Home, level selection, collection, settings, pause, and results are usable without a mic.	End-to-end Touch run through all holes.
FR-02	Permissions are requested in context after Voice is selected.	Fresh-install grant, deny, and permanently-denied recordings.
FR-03	Calibration distinguishes room noise, soft input, and comfortable strong input.	Synthetic tests plus device session records.
FR-04	The power shown on release equals the power used by physics.	Logged shot command and UI value differ by at most 0.01.
FR-05	Invalid/no-signal capture, cancel, and interruption never spend a stroke.	State-transition tests.
FR-06	One accepted release produces one impulse and one stroke.	Double-event and multi-touch regression tests.
FR-07	Touch and Voice use the same normalized power-to-impulse curve.	Same-power trajectory comparison on one device/build.

ID	Requirement	Acceptance evidence
FR-08	A settled, supported ball can be aimed; a moving ball cannot be shot again.	Flat, ramp, and bumper tests.
FR-09	Out-of-bounds recovery uses a verified safe anchor with one penalty.	Repeat fall and blocked-anchor tests.
FR-10	Cup completion requires eligible proximity, height, and low speed.	Near-miss, flyover, and high-speed pass tests.
FR-11	Stars and best strokes persist without duplicate rewards.	Replay and interrupted-save tests.
FR-12	Sequential hole unlocks depend on completion, not three-star mastery.	Low-score progression test.
FR-13	All twelve holes have meaningful layout/mechanic differences.	Content review with solution evidence.
FR-14	Cosmetic choices never change physics.	Collider and tuning equality tests.
FR-15	Pause, app background, focus loss, route change, and input-mode change safely cancel capture.	Physical-device lifecycle matrix.
FR-16	No raw microphone audio is written to disk, transmitted, or included in telemetry.	Storage/network inspection and code review.
FR-17	Settings expose input mode, recalibration, music, effects, haptics, reduced motion, and quality.	Screen and persistence tests.
FR-18	Invalid content or save data fails safely with recovery UI.	Corrupt-file fixtures.
FR-19	Scene loading and back navigation cannot leave two active sessions or microphones.	Thirty enter/exit cycles.
FR-20	Shipped interface components replace, rather than screenshot, the reference art.	Visual review and node inspection.

7. Scoring and progression rules

A committed shot adds one stroke. An out-of-bounds event adds one additional penalty stroke, exactly once for that shot. Scores are evaluated after the ball resolves: a ball can finish on the twelfth stroke; exceeding the limit cannot earn stars. At twelve strokes without completion, show Attempt Finished with Retry and Map. At eleven strokes, a shot followed by a fall resolves as thirteen and fails.

A completion within the limit earns three stars at or below par; two stars at par + 1; one star above par + 1. Store the best star result and the fewest completed strokes independently. A failed attempt does not replace a prior best. No timer-based score, voice-quality score, “best whisper,” accuracy percentage, gem reward, or loudness bonus is included.

Unlock CC01 initially and each next hole on completing the previous hole. PP01 unlocks after CC06; do not gate it on a star total. Show 0–18 stars per world. Finishing PP06 unlocks free replay and the Robot cosmetic. There is no consumable attempt counter.

8. UX and accessibility requirements

The course and landing area have priority over decorative UI. In gameplay, use a compact header and a thumb-reachable bottom control tray; do not repeat the oversized logo from the concept images. The first playfield should not hide the flag behind the power panel.

Use a 390 × 844 logical portrait design reference, flexible containers, explicit safe-area margins, and minimum 48-logical-unit interaction targets as our internal design standard. These are design choices, not claims that Godot uses Android dp automatically. Map device safe areas into the actual viewport transform. Test narrow phones, tall phones, and tablet proportions.

Voice states must be indicated by text/icon as well as color. A visible Listening indicator accompanies capture; a mic icon alone is insufficient. Calibrated power is relative to the device; never label it as real-world sound pressure in decibels. The strongest shot must be reachable at a comfortable voice level.

All holes and rewards must be reachable in Touch mode. Reduced Motion suppresses screen shake, camera overshoot, and excessive celebrations. Do not rely on audio-only instructions or vibration-only feedback. Prefer large icons and concise copy; UI text is editable and localized.

9. Nonfunctional targets

These are proposed budgets to validate, not measured results. Aim for sustained 60 fps on the selected mid-tier Android target and a stable 30 fps quality tier on the minimum supported target. Physics stays at 60 ticks per second across quality tiers. Record p50/p95/p99 frame times and long-run temperature behavior; average FPS alone is insufficient.

Initial budgets: at most 150,000 visible triangles, 100 visible draw calls, 32 MB compressed content per world, one shadow-casting directional light, and no dynamic global illumination. Start with a 150 MB installed-content budget and 450 MB peak process-memory target, then revise based on measured device data. Do not treat arbitrary budget numbers as engine limits.

Voice target: a visible power response within 120 ms p95 of usable incoming samples and a committed shot on the next physics tick after an accepted release. Hardware capture latency must be measured separately. Level-to-level transitions should avoid blocking hitches and release unused resources. Final minimum Android/API/ device support remains a release gate until the device matrix is selected and measured.

10. Privacy and resilience

The game must work offline after installation. Voice analysis is local, temporary, and limited to explicit interactions. Store only calibration numbers and user settings, not recordings. Do not put PCM, transcripts, amplitude timelines, hardware identifiers, or permission decisions into production analytics. The MVP has no network analytics SDK.

Stopping GDScript analysis is not proof that the operating system released the microphone. A specific device gate must verify the hardware privacy indicator and audio-driver behavior after release, pause, and background. If Godot's built-in capture cannot meet this requirement, document the reproduction and evaluate a narrowly scoped native input adapter. Do not hide this issue by relabeling the UI "Mic off."

Save at completed holes, collection changes, and settings changes. A killed application resumes at the last completed progress state; mid-shot resume is not required. The player may lose the current attempt, never previously completed holes. Corrupted primary saves should recover from the last validated backup where possible.

11. Success and validation plan

Run a formative usability session with at least eight representative testers; this is a discovery sample, not statistical proof. Proposed acceptance targets: seven can take an intentional first shot without spoken coaching, six can intentionally distinguish gentle and strong power, and everyone can find Touch fallback after denial. Capture observations, not user voice recordings.

Compare replay enjoyment and frustration qualitatively between Touch and Voice. If voice feels less predictable, fix calibration and feedback before adding content. A visually impressive screenshot does not compensate for unreliable input.

12. Release gates and ownership

Gate G0: pin tools, identify device tiers, validate capture lifecycle and renderer/physics behavior. Gate G1: one complete graybox hole playable in Touch and Voice. Gate G2: one polished vertical slice approved against the reference direction. Gate G3: all twelve holes, three cosmetics, and local progression complete. Gate G4: device, content, save, accessibility, and release checks pass.

Product/design owns mechanics and scope; gameplay owns the shot/physics loop; audio/platform owns capture and permissions; art owns reusable assets; QA owns reproducible evidence. A small team may combine roles but must not omit their acceptance work.

No delivery dates or market outcomes are implied. `tasks/` contains the proposed work breakdown, dependencies, and completion criteria. Any later addition of accounts, ads, purchases, children's targeting, or audio sharing requires a new product and privacy review.

Game Design and Rules

1. Design pillars

Precision before loudness. Every course must reward at least one deliberately restrained action. A stronger voice does not grant a better score; it supplies a different physical tool.

Readable physical play. Give the player a clear launch point, a visible or discoverable landing area, and an understandable obstacle. No foreground foliage on collision edges. Interactive hazards use consistent silhouettes and feedback.

Playful recovery. Falls are short, comprehensible setbacks, not punishment screens. Retry is immediate, free, and does not reset world progress. Celebrate smart short shots as much as long jumps.

Small authored spaces. Build compact islands from a modular kit. Distant islands are scenery, not mandatory simulated geometry. A complete course should be understood through one camera overview.

2. Shot interaction

1. In Ready, drag on the playfield to change horizontal aim; the bottom tray remains stationary.
2. Hold the mic button to enter CaptureWarmup, then Capturing after samples are valid. Lock aim for the hold. The pointer that started the interaction owns it.
3. Speak softly or comfortably strongly. Show a live level plus a labeled “Shot power” preview. Only the preview becomes the shot.
4. Release within the active hit region to commit. Drag into the explicit cancel region and release to abort. A hold above three seconds cancels with “Try a short sound.” It never fires automatically.
5. Consume the immutable ShotCommand on the next physics tick, then disable shooting until supported and settled.

No signal, a canceled gesture, losing focus, or an audio-route change returns to Ready with zero additional strokes. During Capturing, camera gestures and other shot controls are disabled; Pause and Cancel remain available.

Touch mode uses drag-to-aim plus a 0–100% slider and Shoot. Zero power cannot fire. An accessibility alternative to hold is a tap-to-start, tap-to-confirm voice interaction with a clear Cancel button; introduce only after the default loop passes tests. Both paths use the same ShotCommand and stroke rules.

3. Course mechanics

Mechanic	MVP behavior	Feedback
Turf	Predictable rolling surface with authored resistance.	Soft roll sound; grass color.
Rail/wall	Solid bank surface; no hidden collision protrusions.	Brief collision sound and squash accent.
Ramp	Transfers horizontal momentum into a jump through geometry.	Clear chevrons and visible landing platform.
Moving gate	Fixed-period kinematic bumper; timing affects route.	Visible travel range; no lethal spikes in early holes.

Mechanic	MVP behavior	Feedback
Portal pair	Moves ball to paired exit once, transforms direction, preserves allowed speed.	Matching color, arrows, short pulse.
Bounce pad	Adds a configured one-shot impulse on entering.	Compression then bounce cue.
Cup	Low-speed entry captures ball and ends attempt.	Distinct ring, flag, completion animation.
Void	One penalty; reset to last safe rest anchor.	Short fall/fade; "+1 penalty" once.

Avoid moving platforms as resting surfaces in MVP. Avoid physics-driven chains or destructible meshes. Curved art ramps may use simplified segmented collision surfaces with no seams large enough to trap a ball.

4. Rules and numerical tuning

The first tuning pass uses one world unit as one meter, a 0.25 m sphere radius, 1.0 kg mass, gravity 9.8 m/s², and a 1.5–10.0 N·s impulse range. These are starting values, not realistic golf simulation or tested balance.

Normalized power p is in $[0, 1]$. Reject $p \leq 0$. For accepted p , impulse magnitude is $1.5 + (10.0 - 1.5) * p^{1.6}$. Apply once along a normalized horizontal direction tangent to the world XZ plane. Ramps and pads create vertical movement; the standard shot adds no arbitrary upward boost.

Power curve tuning must be versioned with content. Re-run authored solutions after changing ball radius, collision, mass, resistance, physics engine, impulse curve, or physics tick rate. Cosmetics cannot override these parameters.

Par is authored from playtests. Initial pars in the content catalog are proposals. The reference solution need not be the only solution. Do not promise exact cross-platform physics determinism or replay by input alone.

5. Rest, failure, and completion

Ready requires translational speed below 0.06 m/s, angular speed below 0.3 rad/s, stable support on allowed static terrain, and 0.4 seconds of continuous settlement. The system must not freeze a ball that is still descending or on an ineligible slope. Minor cosmetic animation can continue after physical rest.

A terminal fall occurs in the kill volume or below the authored kill plane. A watchdog also offers recovery if the ball remains moving with no useful progress for 20 seconds; it does not silently teleport a possibly valid long roll. Player-confirmed stuck recovery uses the same +1 penalty rule. Timeout during background is not a fall.

For cup capture, require horizontal center distance ≤ 0.28 m, ball center within the authored cup height band, and speed ≤ 1.2 m/s. Test these candidate thresholds with actual cup geometry. Do not let a ball passing high above the cup win. Completion owns its own idempotence key and suppresses any subsequent fall signal from the celebration animation.

6. Level progression

Cloud Cliffs: CC01 First Putt; CC02 Soft Landing; CC03 Bank Buddy; CC04 Little Leap; CC05 Gate Timing; CC06 Cliff Combo.

Portal Peaks: PP01 Through the Ring; PP02 Exit Angle; PP03 Spring Step; PP04 Portal and Pad; PP05 Gate Escape; PP06 Royal Route.

Each level card in `content/level_catalog.json` records the teaching goal, layout brief, route alternatives, common failure, and validation checklist. Three internal graybox diagnostic scenes are also required: straight roll distance, ramp/CCD collision, and cup capture. These are engineering tools, not additional advertised holes.

7. Camera and emotional feedback

Use a consistent three-quarter perspective with the ball in the lower-middle playable zone and the target in the upper area. Ready camera moves slowly and stops before capture begins. Follow mode tracks rolling without hiding the next relevant surface. Overview is a deliberate inspection action, not a mandatory free-camera skill.

A face mesh or decal may be presented toward the camera independently from the sphere's rolling transform for readability. This is a cosmetic convention; the collision body still behaves as a sphere. Avoid rotating a large protruding mane collider with every bounce.

Provide quiet, short sonic feedback. Suppress shot-button clicks, voice playback, and music during capture so the game does not power itself. The lion's roar sound plays only after input has been closed. No screen shake is necessary for ordinary putting.

8. Balancing review

A hole fails design review if a newcomer cannot distinguish the route from background scenery, if perfect loudness is required to cross an unavoidable gap, if the cup is hidden by HUD, if one missed shot causes an inescapable position, or if touch users cannot earn the same reward.

Record minimum/recommended/maximum successful power bands for major shots, not only a single ideal value. For introductory holes aim for forgiving bands around at least ± 10 percentage points in the primary route, subject to playtesting. Later holes can ask for more accuracy but must offer understandable feedback.

Technical Architecture

1. Stack and boundaries

Use the standard Godot 4.7.2 editor/export templates and typed GDScript. Start with integrated Jolt, not an external Jolt addon [S01, S03]. Godot renders both world and UI; there is no Flutter dependency. Build authoring assets in Blender and export `.glb` for reproducible import [S11]. Kotlin is introduced only after a platform requirement passes the decision gate in document 09.

Pin engine version in `ENGINE_VERSION`, CI, export templates, and the contributor guide. A version bump is a tested change, not an automatic update. The exact Android SDK/Gradle requirements must be captured from the selected export template at implementation time [S09].

2. Proposed repository structure

```
project.godot
ENGINE_VERSION
scenes/app/           # boot, loading, home, map, settings
scenes/game/          # gameplay root, ball, camera, cup, HUD
scenes/levels/        # CC01.tscn ... PP06.tscn
scenes/obstacles/     # reusable gate, pad, portal
scripts/domain/       # shot math, scoring, save migrations
scripts/game/         # session controller and scene adapters
scripts/audio/        # VoiceInputService and calibration
scripts/platform/     # GodotPlatformAdapter; native fallback
resources/            # themes, tuning .tres, catalog, cosmetics
assets/models/        # imported .glb; no baked UI text
assets/audio/         # original/licensed music and effects
assets/localization/  # keyed strings
addons/              # empty unless an approved dependency exists
android/plugins/      # absent until a Kotlin decision is approved
tests/unit/
tests/integration/
tests/fixtures/
tools/
```

Keep imported art separate from hand-authored wrapper scenes. A re-export must not erase collision, signals, or spawn markers. Use Git LFS for large art sources once the team chooses its repository policy; do not commit engine caches or signing secrets.

3. Runtime responsibilities

AppRouter owns top-level transitions and one active **GameSession**. **SettingsStore** owns user preferences and calibration. **ProgressStore** owns completed results and cosmetics. **AudioDirector** owns music/effects ducking, not microphone permission. These may be small autoload services; do not make every gameplay node global.

GameSessionController is the sole shot/stroke authority. **InputCoordinator** transforms touch/voice into intent. **VoiceInputService** owns capture buffers and explicit listening state. **BallController** applies accepted impulses and reports settlement. **LevelController** owns spawn anchors and win/fall routing. **CameraRig** consumes session state. **HUDPresenter** displays immutable view data and emits UI intent.

The UI never writes ball velocity, changes stroke totals, or saves best scores directly. A portal cannot increment score. The microphone service never launches the ball. This separation keeps repeated signals and platform callbacks from becoming duplicate shots.

4. Scene composition

```
GameRoot (Node)
  GameController (Node)
  InputCoordinator (Node)
  WorldRoot (Node3D)
    LevelInstance (Node3D)
      Visuals (Node3D)
      StaticCollision (Node3D)
      Spawn (Marker3D)
      Cup (Area3D + visual cup)
      SafeAnchors (Node3D)
      KillVolume (Area3D)
      Obstacles (Node3D)
    Ball (RigidBody3D)
      CollisionShape3D (sphere)
      VisualPivot (Node3D)
    CameraRig (Node3D)
      Camera3D
    Lighting (Node3D)
  GameUI (CanvasLayer)
    SafeAreaRoot (Control)
      HUD (Control)
      PauseLayer (Control)
      ResultLayer (Control)
```

Use `Control` containers and theme resources for UI rather than a single composited mockup PNG [S12]. Place scene art under `Node3D` and labels/buttons under `CanvasLayer`. Collision ownership belongs to wrapper scenes, not arbitrary mesh names.

5. State machine

Top-level states: Boot, Home, Loading, Playing, Results, RecoverableError. Inside Playing: Intro, Ready, CaptureWarmup, Capturing, CommitPending, Rolling, Settling, Resetting, Paused, Complete, Failed.

Current	Event / guard	Next / effect
Ready	valid mic press, consent/calibration ready	CaptureWarmup; allocate session token, lock aim
CaptureWarmup	first fresh frames, still held	Capturing; show Listening
Capturing	valid release and stable signal	CommitPending; close capture; queue one ShotCommand
Capturing	cancel, timeout, focus loss, route change	Ready or Paused; close capture; no stroke
Ready	positive touch power + Shoot	CommitPending; queue same command type
CommitPending	valid session token at physics tick	Rolling; +1 stroke, apply one impulse
Rolling	below thresholds with support	Settling
Settling	stable duration reached	Ready; update safe anchor
Rolling / Settling	fall	Resetting; +1 penalty once

Current	Event / guard	Next / effect
Rolling / Settling	eligible cup entry	Complete; persist completion once
Resetting	verified reset finished, limit not exceeded	Ready
Any playing state	pause/background	Paused; preserve simulation snapshot, cancel uncommitted shot
Paused	resume	prior non-capture state, never auto-restart mic

An accepted but not yet applied command is canceled if background/pause happens before the physics tick. Once applied, the stroke remains spent. Resume a Rolling ball with its paused state, not a fresh impulse. Intro/Ready/Capturing cannot win from stale cup signals.

6. Data contracts

ShotCommand : session_id, shot_id, source (voice/touch), normalized_power, direction_world, level_id, tuning_version. The main thread creates it after validation. No PCM or microphone identity is attached.

ShotResolved : shot_id, outcome (rest/cup/out_of_bounds), resting_transform, strokes_total. Resolution accepts each shot_id only once. Compound signals from kill volume and kill plane coalesce.

LevelResult : completion_id, level_id, content_version, completed, strokes, par, stars, elapsed_active_seconds. Time is informational, not a rank. ProgressStore rejects duplicate completion IDs and does not overwrite a better result.

VoiceStatus : capture_id, phase, relative_live_level, preview_power, signal_valid, error_code. Errors include permission_denied, no_input, too_noisy, route_changed, interrupted, timeout, and buffer_overrun. UI maps error codes to localized copy.

7. Dependency and event policy

Use typed signals for one-way events and explicit methods for commands. Pass dependencies through constructors/setup methods or exported node references where practical. Avoid stringly typed global event buses. Never connect the same handler on each scene re-entry without disconnecting it.

Main-thread code owns the scene tree. DSP over short buffers should remain bounded; introduce a worker only after profiling. Native callbacks must marshal safely to Godot's expected thread rather than mutate scenes from an Android audio callback. No per-frame Kotlin bridge messages are needed for default capture.

8. Loading, saves, and failure recovery

Validate catalog data before displaying locked levels. Map validated level IDs to a packaged allowlist of .tscn resources. Do not instantiate arbitrary paths from a save file. Clear the previous LevelInstance, capture, pending commands, and listeners before loading a new session.

Read local settings/progress using a versioned schema, strict type/range validation, and a last-good backup. FileAccess and JSON are supported building blocks, not an automatic transactional database [S13]. See document 08 for replacement/recovery rules.

If loading fails, show Retry and Home. Do not show a fake course or silently unlock the next hole. Errors should include a non-sensitive diagnostic code usable by QA.

9. Rendering approach

Choose the Mobile renderer as the initial 3D candidate, then compare with Compatibility on target devices [S02]. Keep visual requirements within a shared, deliberately modest feature set: static lighting where possible, simple PBR/stylized materials, one main shadow light, modest particles, and cheap waterfall ribbons.

Quality tiers may change resolution scale, particles, foliage, and shadows. They must not change collision, input timing, physics tick rate, or scoring. Changing renderer is a tested configuration change; do not assume that a live low/high quality toggle can replace the rendering backend safely.

10. Testing seams

Pure domain functions cover voice normalization, power curve, scoring, unlocks, and save validation. Integration scenes cover impulse, settlement, cup, portal, and reset behavior. UI tests cover safe area, focus, selected/disabled states, and screen navigation. Device tests cover rendering, latency, native privacy indicators, and interruptions.

This handoff is a specification and reference-code package. It does not include a complete Godot project, imported game assets, or a tested Android binary. Do not mark a task Done solely because a design document or interface stub exists.

Voice Input, Calibration, and Privacy Specification

1. Scope

Detect relative sound level, not words, identity, emotion, or a literal whisper classification. No speech model, transcription service, cloud inference, audio upload, or persistent recording is necessary. The in-game labels Soft and Strong describe calibrated power bands.

Godot exposes microphone playback through `AudioStreamMicrophone` and raw stereo floating-point bus frames through `AudioEffectCapture` [S05, S06]. Input must be enabled in project settings and operating-system permission must be handled [S07, S08]. Hardware/device behavior must be verified rather than inferred from desktop success.

2. Bus and lifecycle design

Proposed routing: `MicPlayer` -> `MicCapture` bus (`AudioEffectCapture` first) -> `MicSilence` bus -> `Master`. Silence the monitoring path after the capture effect, never the `Master` bus. Confirm on devices that the capture effect still receives frames and the microphone is never played through the speakers. Do not apply gain/limiting before measurement unless explicitly modeled in calibration.

`MicPlayer.autoplay = false`. Capture starts only through `VoiceInputService` after the user explicitly selects Voice/calibration or starts a shot interaction. Allocate a fresh capture token, clear stale frames, duck game audio, start playback/capture, and wait for fresh frames. UI says “Starting mic” until frames arrive; only then says “Listening.”

On release/cancel/pause/background/route change, stop capture playback, stop processing, clear the ring buffer and rolling level history, invalidate the token, and restore output audio after shutdown. Never restart automatically on resume. Test the OS microphone indicator separately: stopping analysis or muting a bus is not equivalent to releasing the input device.

If the engine/driver leaves input active outside the expected capture period, fail the privacy gate. Investigate the selected engine's driver behavior first. A native Android audio adapter is permitted only after a documented need; it must acquire/release the actual recorder, and the built-in path must then be disabled to prevent two microphone owners. See document 09.

3. Capture and computation parameters

Starting values (all configurable): warmup discard 100 ms; frame analysis windows 20 ms; capture ring 250 ms; power smoothing time constant 80 ms; recent commit window 250 ms; minimum qualified duration 150 ms; maximum hold 3,000 ms; no-fresh-samples timeout 500 ms.

For `AudioEffectCapture`, determine the frame rate of its bus stream from the active mixer rather than hardcoding a device input rate. Compute window sample counts from that rate. The engine exposes rate-related APIs; input hardware and output mix rates may differ [S07]. The buffer API consumes frame counts and can report discarded frames [S06].

For stereo frame i , energy is $(\text{left}_i^2 + \text{right}_i^2) / 2$. Window RMS is $\sqrt{\text{mean}(\text{energy})}$. This avoids cancellation from averaging out-of-phase channels before squaring. Convert to relative digital dBFS as $20 * \log_{10}(\text{rms} / 0.000001)$. Internally clamp to $[-120, 0]$. UI must not display this as measured real-world dB SPL.

Reject non-finite samples. Clamp only for numeric safety; count clipping separately when either channel magnitude ≥ 0.98 . A capture overrun invalidates the current shot rather than making old audio appear current. Do not retain more than the small bounded window required for the algorithm.

4. Calibration procedure

Calibration is a skippable three-stage interaction plus a test shot. Soft/strong requests must say “Use a comfortable voice; no shouting needed.” Never reward longer or louder calibration.

Stage A: after permission, capture 1.5 seconds of room noise while the player stays quiet. Use the 90th percentile of valid window dBFS as `noise_db`. Stage B: capture a comfortable soft sound for up to 1.5 seconds and compute the median qualified window level as `soft_db`. Stage C: capture a comfortable stronger sound for up to 1.5 seconds and use its median as `strong_db`. Stop early only when enough qualified samples are stable.

Candidate quality checks: `soft_db >= noise_db + 6`, `strong_db >= soft_db + 8`, no excessive clipping ($>5\%$ of samples), and at least 0.3 seconds of qualified soft and strong data. These are proposed thresholds, not universal microphone truths. If failed, offer Recalibrate, Move closer/choose a quieter place, or Use Touch. Do not require repeated escalating vocal effort.

Set `gate_db = noise_db + 4`, `lower_db = max(noise_db + 6, soft_db - 3)`, and `upper_db = strong_db`. Store only these numbers, calibration algorithm version, and a non-sensitive route category (built-in/wired/Bluetooth/unknown). Never store a hardware identifier. Recalibrate on route change and when signal/noise behavior changes significantly.

5. Runtime power mapping

For an eligible window above the gate, compute `raw = clamp((dbfs - lower_db) / (upper_db - lower_db), 0, 1)`. If the denominator is invalid or too small, require recalibration and do not shoot.

Smooth with frame-time-correct exponential interpolation: `alpha = 1 - exp(-dt / 0.08)`; `smooth += alpha * (raw - smooth)`. A longer frame must not make control radically slower. Clamp smooth to $[0, 1]$. This is relative power input, not yet the physical impulse curve.

The **Shot power preview** is the 75th percentile of qualified smoothed windows in the most recent 250 ms. Require at least 150 ms of qualified windows within that window and `preview > 0`. The same value is displayed and committed. A live amplitude bar may coexist but must have a different label. If the player stops making sound, preview expires as qualified samples age out; do not keep an old maximum forever.

On pointer release, use the last published preview from the same capture token. Stop/clear input and enqueue one ShotCommand. The UI does not pretend to account for unobserved device latency. Playback of the lion's sound occurs after capture closes.

6. Interaction edge cases

Case	Required outcome
Permission is pending	Disable capture; offer Touch; no stroke.
Permission denied	Explain without nagging; persist Touch preference.
Permission permanently denied	Offer app settings plus Touch; no prompt loop.
All-zero samples	"No microphone signal"; do not interpret as a quiet shot.
Device too noisy	Cancel invalid shot and offer recalibration/Touch.
Phone tap / single clap	A transient shorter than qualified duration cannot shoot.
Sustained background sound	May resemble voice; disclose amplitude-only limitation and offer Touch.
Strong input clips	Saturate display safely; advise gentler input; never exceed maximum impulse.
Multitouch	Only initiating pointer controls capture; secondary release cannot fire.
Background or incoming interruption	Cancel uncommitted shot; close input; never auto-fire on return.
Bluetooth route changes	Cancel; invalidate calibration; no simultaneous input owners.
Release arrives twice	Token and shot-ID guard accept at most one command.

Do not claim amplitude processing rejects speech from other people or knows which sound is the player's. This is a limitation to communicate and test, not hide with an "AI noise filter" label.

7. User-facing copy

Permission pre-prompt: "Use your sound to set shot power. Sound is processed on this device and is not saved or uploaded. You can play with touch instead." This copy may ship only after implementation proves it.

Calibration prompt: "First, stay quiet. Then try a soft sound and a comfortable stronger sound." Gameplay: "Hold, make a sound, release to shoot." Cancel: "Slide here to cancel." No signal: "We couldn't detect a usable sound. Try again or use touch."

Do not say "Mic off" until the native release behavior has been validated. Accessibility copy should not imply that a speech impairment is a player failure. A soft hum or other comfortable sound can be used; no word matching is involved.

8. Synthetic and device validation

Synthetic fixtures must cover silence, fixed amplitudes, opposite-phase stereo, clipped values, short impulse noise, irregular frame times, too-small calibration ranges, stale frames, and duplicated release events. Calibration tests use digital values, not invented acoustic measurements.

Device validation must include room noise, phone speaker playback, headphones, permission revocation, repeated holds, a thirty-cycle input stress test, background/foreground, route changes, and microphone privacy indicator observation. Report measured capture/UI latency with the device/build/route. No physical microphone or Android audio path has been tested in this package.

Physics, Obstacles, and Camera Specification

1. Physics setup

Use one `RigidBody3D` ball with a `SphereShape3D` collider, integrated Jolt, fixed 60 Hz physics, and continuous collision detection enabled for the fast-moving ball. Godot documents impulse-based rigid-body control and CCD behavior [S03, S04]. CCD reduces missed collisions; it is not a guarantee against every tunneling case.

Start with radius 0.25 m, mass 1.0 kg, and gravity 9.8 m/s². Keep visual scale independent of collision scale. Avoid nonuniformly scaling the physics body. A decorative lion mane cannot protrude into a larger gameplay collider than other cosmetics.

Use static collision for course surfaces, kinematic/`AnimatableBody3D` wrappers for moving obstacles, and `Area3D` triggers for cup, pads, portals, and kill volumes. Complex scenery must not accidentally become collidable. A fixed collision-layer convention is part of the project setup: Ball, Course, MovingObstacle, GameplayTrigger, and CameraBlocker.

2. Impulse and movement

An accepted shot becomes `direction.normalized() * impulse_magnitude`. Apply once via the rigid body's impulse API on a physics tick. Do not apply the same impulse every rendered frame or multiply an impulse by delta. Do not directly overwrite position each frame to imitate a rolling simulation [S04].

At Ready, remove only measured numerical drift; do not reset a still-moving body's angular motion to force readiness. The initial impulse curve is the shared domain function in document 02. The candidate upper speed cap is 18 m/s for bug containment after pad/portal interactions. If cap use becomes frequent, fix tuning rather than silently flattening every strong shot.

Use modest physical friction and explicitly tune rolling resistance. Friction alone is not a reliable design specification for how far a golf ball should roll. Proposed turf resistance decelerates tangential speed at 0.55 m/s² while supported; wood uses 0.35 m/s². Compute only the tangential component relative to contact/support normal. Preserve gravity and vertical momentum; do not apply ground resistance in flight.

Avoid double-counting resistance through both heavy engine damping and a custom term. An initial small global damping of 0.05 can be evaluated; keep a tuning log. Reduce tangential speed toward zero, never reverse it. The implementation belongs in the safe physics integration path, with tests on flat terrain and slopes.

3. Support and resting anchors

A rest detector checks speed, angular speed, support, and duration. The authored thresholds are 0.06 m/s, 0.3 rad/s, and 0.4 seconds. Require a valid static support with a normal close enough to up for a safe reset position. Proposed maximum safe-anchor slope: 15 degrees. Being almost motionless in midair is not rest.

Store the last safe resting transform, not the ball's last position before a fall. Exclude portals, pads, moving gates, cup capture zones, and positions too close to a cliff. A sphere overlap/sweep verifies free space before reset. If the saved anchor is blocked or no longer valid, use the last authored checkpoint or Spawn marker.

Reset must clear linear/angular velocity and active forces, move through the engine's safe physics-state mechanism, and restore motion without producing a residual impulse. Disable relevant trigger reactions during the reset window, then re-enable after the ball is safely supported. A one-shot penalty token prevents double counting.

4. Cup capture

Keep the cup collision and cup trigger distinct. Model a real opening/rim or intentionally use a visually convincing assisted capture; document the chosen approach. Do not cover a visual hole with invisible flat floor and expect a convincing drop.

The trigger verifies horizontal distance, a center-height band relative to cup surface, and low speed. Candidate thresholds: ≤ 0.28 m horizontal, ball center from 0.15 to 0.40 m above cup plane, speed ≤ 1.2 m/s. Adjust these against the actual 0.25 m radius and rim geometry. High-speed flyovers and underside entries must not complete.

Once accepted, `GameSessionController` enters `Complete` exactly once. Turn off shot input and fall resolution, then use a short cosmetic sink animation. Persist the result before presenting `Next Hole`. A celebratory visual copy of the ball may bounce; the scoring body is no longer active.

5. Portals

Portals are authored pairs with entry/exit transforms and explicit local forward axes. Construct a mapping basis from the entry frame to the exit frame; define the art's facing convention once. Transform velocity direction through that mapping and preserve allowed speed, rather than blindly reversing X or Z.

Place the ball at the exit clearance marker plus a safe radius offset. Check for collision-free space. A paired-portal cooldown is per ball and pair: candidate 0.4 seconds and until the ball has left the exit trigger. Do not rely on time alone while the ball remains inside. Mark teleport as a controlled transform update and ignore stale triggers for the transfer tick.

The exit must be visible in `Overview` and share the pair's symbol/color. Level validation ensures each portal has exactly one valid counterpart, an unobstructed exit, and no unavoidable immediate re-entry loop. Allow at most one pair per MVP hole to control complexity.

6. Bounce pads and moving gates

A pad applies its authored impulse only on a new eligible contact/entry. A contact latch clears after leaving. Holding a ball inside an `Area3D` must not add energy every frame. Use shared tuning with an allowed-speed cap and tests for side entry and repeated bounces.

Move gates using fixed physics-time motion and a clear period/phase. Candidate initial period is 3.0 seconds. Motion pauses with the session, not with wall-clock time; returning from background must not suddenly sweep through the ball. Kinematic collider movement must match the visible obstacle.

No gate should seal the only possible resting location. Gate collision feedback is harmless and readable; it may knock the ball away but must not trap it indefinitely. Avoid elaborate chained rigid bodies for the first release.

7. Camera modes

Ready camera: proposed 45-degree field of view, elevation 45–55 degrees, distance fit to ball and next relevant target. These are starting settings, not mandatory fixed coordinates. Keep the player ball above the bottom tray. Orient world-forward consistently within each hole; camera rotation is not part of the puzzle.

Follow camera: smoothed positional tracking with bounded speed; small look-ahead in travel direction; no automatic spin as the ball rolls. If the ball drops, keep its last useful course view then transition to reset; do not dive into the void after it.

Overview camera: fit authored course bounds excluding remote decorative islands. Freeze aim changes, but do not let overview pause moving hazards only when it benefits scoring; either overview is available only in Ready or explicitly pause the whole session consistently. MVP chooses Overview only in Ready, with simulation paused during inspection.

During Capturing, lock camera and aim so a thumb motion does not accidentally redirect the shot. On narrow screens adjust camera framing, not collision geometry. Reduced Motion uses direct or gentle transitions and no shakes.

8. Aiming visualization

MVP uses a direction ray and short dotted guide projected onto nearby terrain, plus explicit shot power. It is not an exact trajectory simulator. Clip it at the first obstruction and avoid promising a landing point past an unseen collision.

A later predictive preview must run the same physics/tuning model or clearly indicate uncertainty. Do not build a fake parabola that ignores rolling resistance, ramps, gates, or portals. That feature is outside the MVP and is not required to match the reference image.

9. Regression evidence

Each tuning change runs flat roll distances at 25/50/75/100% power, oblique wall rebounds, ramp launch, high-speed thin barrier, pad latch, portal exit/loop protection, cup flyover, slope settlement, safe reset, and duplicate fall events. Save a tolerance band per engine/content version, not an assertion of bit-identical outcomes across all devices.

UI/UX Implementation Specification

1. Source of truth

The seven supplied Roarball PNGs are the existing AI-generated concept references from this conversation. They establish the lion-ball identity, floating islands, saturated warm/cool palette, and rounded game UI. They are not layered assets, real-time screenshots, final text, validated navigation, or measured performance targets.

Use this specification and PRD when a concept image conflicts with the MVP. Do not ship a full-screen mockup PNG with invisible buttons on top. World, camera, lighting, UI, icons, and interactions must be separate implementation elements.

2. Reference reconciliation

Reference	Keep	Change for MVP
01_original_gameplay.png	Mascot, floating turf, clear flag, playful obstacles.	Smaller header; no coins; real shot/cancel states.
02_home.png	Hero scene, dominant Play, friendly brand.	Remove daily challenge, player level, coins; use Play, Map, Balls, Settings.
03_world_map.png	Islands, connected hole nodes, star results.	Only two worlds/six nodes each; remove hearts/shop/lava world.
04_calibration.png	Large mic, simple scale, practice setting.	Real noise/soft/strong steps, Touch skip, cancel/error states.
05_advanced_gameplay.png	Ramps, gates, portals, readable depth.	No wind fan in MVP; never introduce every hazard in one early hole.
06_collection.png	Selected ball pedestal and Equip action.	Lion/Panda/Robot only; no currency/rarity/Trails/Emotes shop.
07_results.png	Celebration, stars, clear Next Hole.	Strokes/par/stars only; no whisper %, accuracy %, gems, or Share requirement.

3. Design tokens and layout

Reference canvas: 390 × 844 logical units. Base spacing: 4; regular gaps: 8/12/16/24. Page margin: 16 plus mapped safe inset. Panel radius: 20; button radius: 16. Small controls have minimum 48 × 48 touch boxes even when visible icons are smaller. Primary buttons are 56–64 high. Mic control is 80 × 80 with an outer cancel-safe gesture area.

Typography target: title 28/34, screen heading 24/30, body 16/22, compact label 14/18. Use a separately licensed rounded typeface or Godot's default during development. No font files are bundled here. Never bake localization text into textures. Avoid all-caps paragraphs and long slogans across scenery.

Palette: dark navy panels #132B3A, panel variant #1D4258, light text #F5F9FC, secondary text #B9CDDA, voice cyan #2DCEFF, primary green #63DC65, warm accent #FFBE42, danger #EF6A6A. Verify final contrast in rendered states; these are proposed tokens, not a completed accessibility audit. Disabled controls need shape/label distinction, not only lower opacity.

Use a reusable Theme resource, StyleBoxFlat/texture-backed nine-patch panels, Buttons, Sliders, Labels, and Container nodes. Godot's stretch and multi-resolution behavior must be configured deliberately [S12]. Do not interpret 390 logical units as 390 physical pixels on every device.

4. Screen inventory and acceptance

UI-01 Boot and loading

Show compact logo, meaningful progress if available, and no mic dialog. Loading failure offers Retry and Home with a diagnostic code. Animated background is optional; never block input with an endless spinner. Do not fabricate percentage progress from an unrelated timer.

UI-02 Home

Hero lion on one simplified tee with distant islands. Top bar: Roarball and Settings, not a faux player account. Primary Play; secondary World Map and Balls. Small input-mode chip says Voice or Touch and opens settings. After first completion, Play opens next unlocked hole; after full completion, Play opens last played level.

UI-03 Mode choice and permission pre-prompt

Two equally visible options: Use Voice and Use Touch. Explain local temporary processing. Continue requests the platform permission only for Voice. Back returns home. Denial does not launch calibration repeatedly. A desktop keyboard testing shortcut is a developer tool, not a mobile onboarding option.

UI-04 Calibration

Header Back and "Find your shot power." Step display: Room / Soft / Strong. Each step includes one sentence, visible duration/progress, a Start action, current signal validity, and Use Touch. A test shot follows valid calibration. No automatic recording begins just because this screen was navigated to.

Display comfortable input range, not a red demand to scream. Invalid signal shows a reason and Retry Step. Keep previous valid calibration until a replacement succeeds. A route change marks old calibration stale without deleting player progress.

UI-05 World map

Two chapter cards or a lightweight vertically scrolling island map. Each has six numbered nodes, earned stars, lock explanation, and a current-level indicator. Each node offers title, par, best strokes, and Play in a small detail sheet. Buttons remain two-dimensional Controls even when the map art looks 3D.

Avoid a heavy active 3D scene per visible node. Use one background/map scene or rendered thumbnails. Navigation: Home / Map / Balls. Locked PP01 explains "Finish Cloud Cliffs" rather than showing a price.

UI-06 Gameplay Ready

Top safe row: Hole name/number, Par, Strokes, Pause. A small objective chip can appear below for the first introduction, then dismiss. Center: course, ball, target. Bottom tray: input-mode chip, power preview, mic or touch slider/Shoot, and Overview. Omit collection shopping and currencies during play.

Aim gestures operate only outside UI hit regions. In Voice mode initial copy says "Aim, then hold to power." Power is visibly zero/unset until usable input. In Touch mode slider preserves its last selected value between shots but requires a new Shoot press.

UI-07 Gameplay Capturing

Same composition, with camera locked. Label Starting mic then Listening; live level and clearly labeled Shot power. Mic ring animates gently. A Cancel region appears above/aside the thumb; release rules are discoverable in tutorial. Other game navigation is disabled, but Pause and Cancel always work. Invalid signal never creates a hidden shot.

UI-08 Rolling, falling, and reset

Power controls are dimmed and labeled “Ball moving.” Do not leave a mic glow that suggests listening. On fall show “Out of bounds · +1 stroke,” then a brief reset without a modal penalty screen. Disable repeated taps during reset. At attempt limit, show Retry and Map.

UI-09 Pause and settings

Pause sheet offers Resume, Restart Hole, Input & Sound, and Map. Restart/exit while an attempt is underway asks for confirmation; completed progress remains safe. Resume never restarts capture. In settings expose Touch/Voice, Recalibrate, volume, haptics, reduced motion, and quality. Input switching mid-hold cancels that hold.

UI-10 Results

Lion celebration above a panel: Hole Complete, earned stars, strokes and par, world completion progress. Primary Next Hole, secondary Retry and Map. Best-result improvement can be labeled New Best. No fabricated accuracy or voice-performance metric. On final hole, replace Next with World Map and show all-holes completion.

The first result write must complete or yield a recoverable Save Warning before navigation. If saving fails, keep the result in memory, show Retry Save, and avoid pretending progress is durable.

UI-11 Collection

One selected 3D preview; three cards with names and unlock criteria. Locked cards are informative, not purchase funnels. Equip changes visuals only. Selected and equipped are separate states: previewing a locked Robot does not equip it. Use a static thumbnail grid, not multiple animated viewports.

UI-12 Recoverable errors

Include designed states for no microphone, denied permission, too-noisy calibration, stale route, save failure, missing level data, and graphics incompatibility. Each state has one clear recovery action and a safe exit. Do not silently fall back to a different input mode while the user's finger is committing a shot.

5. Responsive behavior

At 360-wide phone reference size, collapse secondary labels and keep controls at least the target size; do not scale the entire UI down to fit. On taller phones add course space above the tray. On tablets cap menu content width and use a two-column settings/collection layout. In landscape, move the control tray to the right with course framing adjusted; portrait remains the preferred orientation.

Read the device safe region and convert it into the chosen canvas coordinates before applying margins. Test notches, gesture/navigation bars, and display cutouts on real devices. The UI's minimum hit sizes apply in logical canvas coordinates after the stretch policy, not in untransformed native screen pixels.

6. Interaction and motion

Button press: 80–120 ms subtle scale/color change; screen navigation: 180–250 ms; results: <1.5 seconds before Next is usable. Reduced Motion can remove transforms and shorten celebrations. Do not use long entry animations that lock every tap. These are proposed feel targets.

Back stack is explicit. Android Back dismisses the top sheet first, pauses gameplay second, and never exits mid-capture without cancellation. Home does not request a microphone. A settings toggle must not accidentally pass through to the 3D world.

7. Visual handoff checklist

For every screen, submit portrait and narrow-device captures, default/pressed/disabled/error states where relevant, safe-area overlay, and a short interaction recording. Compare hierarchy, silhouette, color, and affordance to the references; do not demand pixel identity with impossible concept geometry.

A vertical slice is approved only after real buttons, real camera framing, the real ball, real input feedback, and at least one real course are shown together. UI-only reskins of a graybox do not satisfy this visual gate.

Art Direction and Asset Pipeline

1. Translation from concept to real-time

Keep the cheerful lion ball, rich green turf, pale stone cliffs, wood ramps, blue sky, and warm flags. Replace expensive cinematic detail with stylized shape, baked detail, readable silhouettes, and a small modular kit. The reference images are direction, not production assets or a performance promise.

Distant waterfalls use ribbons/planes and lightweight material animation. Distant islands may be low-detail meshes or background cards. Do not simulate water, render an entire kingdom, or use dozens of dynamic shadow lights to copy a concept image.

2. Asset roster

Asset group	MVP contents	Initial budget
Ball	Lion, Panda, Robot; common sphere collider.	2–4k triangles each; 1 material preferred.
Course kit	Straight, corner, wide green, cliff edge, ramp, rail.	0.5–2k triangles/module; shared materials.
Cup kit	Rim, flagpole, flag, bottom/sink visual.	<2k triangles, simple collider.
Obstacles	Gate, portal entry/exit, pad.	1–3k triangles each; collision simplified.
Environment	Rocks, trees, bushes, distant islands, waterfall ribbons.	Instanced repeats; no decorative collision.
UI	Panels, icons, stars, input badges, map thumbnails.	Reusable theme; no baked text.
Effects	Collision puff, pad pulse, portal flash, small confetti.	Bounded particle count; reduced-motion variant.
Audio	Putt, impact, fall, cup, pad, portal, result, ambient music.	Short licensed/original clips; no recorded player sound.

These budgets are proposals. Final visible-scene budget is more important than the sum of every library asset. Record draw calls, material instances, texture memory, and transparency cost on the target device.

3. Blender and Godot workflow

Author using a consistent real-world scale and apply transforms before export. Use one meter per Godot unit. Export GLB/gITF with predictable names, origins, and material paths; this is a supported/recommended Godot import route [S11]. Keep `.blend` source under the team's art-source policy, with exported `.glb` as the runtime interchange artifact.

Store pivot conventions in an asset checklist: ball centered at origin; course module grid alignment; gate hinge/slide axis; portal local forward and exit marker; pad up-axis. Reimport one test asset before committing the whole kit.

Hand-authored Godot wrapper scenes own collision, interaction areas, scripts, sounds, and semantic markers. An artist's re-export must not replace a tested cup trigger or portal link. Use simplified box/convex collision for active objects; detailed background cliff meshes do not need colliders.

4. Material and lighting plan

Start with a shared turf/stone/wood palette atlas and no more than one or two material slots per common prop. Use 512–1024 textures for most props and at most 2048 where justified. Bake surface detail rather than using extra geometry everywhere. Avoid layered transparent foliage covering much of the phone screen.

One primary directional light and baked/static ambient treatment are the baseline. Establish the look on the chosen mobile renderer before producing final materials. Do not rely on unsupported or expensive effects simply because they appear in an editor screenshot [S02]. Low quality removes decorative effects, not gameplay cues.

5. Mascot and cosmetics

Make the ball's face readable at ordinary gameplay size. Use a compact mane integrated into the spherical silhouette; protruding horns and ears remain cosmetic. Panda and Robot change materials/face shells, not collider shape or mass.

Required expressions: idle, concentrating, rolling, surprised, success. Start with simple blend/material swaps or transform-based expressions instead of a complex rig. Optional face stabilization should not alter the physics transform.

6. UI construction

Rebuild panels and controls in a theme. Export only original icons and decorative textures as transparent assets with padding for nine-patch use where needed. Never crop a screenshot's button including text and use it as the final control. All text remains real UI text for localization and scaling.

`design/design_tokens.json` is a machine-readable starting point, not a generated Godot Theme. A task explicitly implements and verifies the theme. Fonts must be selected/licensed by the implementation team; no font binaries are included in this delivery.

7. Asset acceptance and provenance

For each asset, record author/source, license, modifications, source file, export file, triangle/material count, collider owner, and usage. The supplied AI concept images remain references; they are not proof of trademark clearance, exclusive rights, or a third-party asset license. The working name Roarball has not been cleared for commercial use.

Before approving a level, review it at actual phone size and in low quality. Collision boundaries, cup, gate timing, and portal pairing must remain readable without visual effects. Artifact files in this package contain no final `.glb` meshes or production audio.

Content, Local Saves, and Data Validation

1. Content approach

Author geometry as Godot scenes with modular reusable objects. Use a small JSON catalog for ordered metadata and build-time validation. This package includes twelve metadata briefs and a JSON Schema. It does not include the twelve playable scenes; the scene paths are the target contract for implementation.

The application maps a validated catalog ID to a packaged scene allowlist. Saves reference IDs only, not arbitrary scene paths or scripts. JSON metadata cannot invoke methods, choose classes, or load network content. IDs are permanent once released.

Required level scene markers: Spawn, Cup, KillVolume or kill plane config, CourseBounds, and at least one safe fallback anchor. Optional obstacles expose stable IDs so validation can check portal pairs and references. Each hole has at most one portal pair in MVP.

2. Catalog fields

Each level entry has `id`, `world_id`, `order`, `title`, `scene_path`, `par`, `max_strokes`, `mechanics`, `teaching_goal`, `layout_brief`, `solution_brief`, `validation_checks`, and `status`. Status is `design_only` in this delivery. A built level may become `graybox`, `playtest_ready`, then `validated` with evidence stored separately.

Schema validation enforces types/ranges. Semantic validation additionally enforces uniqueness, contiguous order, exactly six levels per world for this release, twelve total levels, known mechanics, matching path/ID, and `max_strokes >= par + 2`. A schema passing does not prove a course is fun or solvable.

Reference solutions record an engine version, tuning version, level scene/content hash, device/build, aim/power bands, stroke count, and notes. Do not store only an ideal float sequence and call it deterministic playback. A human tester must independently reproduce a successful route.

3. Local save format

Store local progress separately from settings. Use schema version 1 initially. Proposed progress envelope:

```
{
  "schema_version": 1,
  "content_version": "mvp-1",
  "best_results": {
    "CC01": {"best_strokes": 2, "best_stars": 3}
  },
  "completed_levels": ["CC01"],
  "equipped_cosmetic": "lion",
  "last_played_level": "CC02",
  "applied_completion_ids": ["local-session-id:CC01:1"]
}
```

Unlocks are derived from completed IDs and current catalog order, not trusted free-form booleans. Validate positive integer strokes, star range 1–3, allowed cosmetics, and known IDs. Do not allow malformed data to equip an unearned cosmetic or crash the game. Preserve unknown future fields only through an explicit migration policy, not blind dictionary merging.

Settings store input mode, volume, haptics, reduced motion, quality, and calibration constants/version. Do not store PCM, a waveform history, audio recordings, transcripts, user names, or hardware device identifiers.

4. Write and recovery algorithm

Godot's FileAccess and JSON APIs are appropriate primitives for local persistence [S13]. The game still needs explicit validation and recovery.

1. Build an immutable new save snapshot in memory and validate it.
2. Write a temporary file in the same `user://` directory. Flush and close; re-read and validate the bytes/JSON.
3. Keep the previous validated primary as the last-good backup. Replace the primary through a checked rename/replacement operation appropriate to the platform. Do not delete the only valid copy first.
4. On any write/rename error, retain the in-memory snapshot and previous durable version; show Save Warning with Retry Save.
5. On boot, validate primary; if invalid, validate backup. Recover the last valid one and inform the player. If neither is valid, offer a fresh local profile without pretending the old data was recovered.

Test failure at every step. “Atomic” must not be claimed universally from a desktop test; mobile process-kill and filesystem behavior require validation. Temporary files must never be trusted automatically without schema validation.

5. Completion idempotency

Generate one completion ID per completed attempt. Apply the result once even if cup trigger, animation callback, and result screen all fire. Better stars and lower strokes update only on a valid completed result. Limit stored completion IDs to a bounded recent set sufficient for local duplicate prevention; better-result merging makes older replayed duplicates harmless because MVP has no currency.

Completed-level membership never shrinks because of a worse replay. Cosmetic unlocks are derived from CC06 and PP06 completion. At max-stroke failure, no completion entry is written. Save settings independently to avoid reverting progress when input preferences change.

6. Versioning and migration

Every engine/physics/tuning change records a version. Existing best strokes may remain as historical personal results, but content designers must decide how materially changed holes affect comparison. Never silently reset all player progress because an obstacle was moved.

A future schema migration must back up old data, transform to a new object, validate, then commit. Unit fixtures cover missing optional settings, unknown level IDs, invalid numeric values, duplicate completions, future schema version, and truncated JSON. Future schema version should produce a safe unsupported-save state instead of guessing.

7. Diagnostics

Use developer-only logs with level IDs, state transitions, normalized shot power, and non-sensitive errors. Normalized power can be logged in controlled QA builds without any PCM; disable histories in production. Never include capture buffers or full device identifiers in crash attachments.

The MVP has no server schema, authentication, leaderboard, or analytics event collection. A future online layer must have its own security, consent, data-retention, and abuse design; it is not a hidden prerequisite of the local game.

Android Delivery, Optional Kotlin, and Build Runbook

1. Default: no Kotlin module

Implement gameplay, HUD, calibration logic, input state, saves, and scene flow in typed GDScript. The normal Godot Android export workflow does not require the team to build a separate Kotlin app [S09]. Do not create a Flutter host or Android Activity wrapper just to keep a familiar language in the stack.

Kotlin becomes eligible only for a demonstrated Android capability gap: microphone acquisition/release control that the built-in path cannot satisfy, a required native settings/deep-link action unavailable through a supported API, or a later explicitly approved Android SDK integration. Ads, billing, notifications, and Play services are not MVP requirements.

2. Native-integration decision gate

First create a minimal reproduction on the pinned Godot version and two physical Android devices. Record expected behavior, observed driver/permission/lifecycle behavior, and why the supported GDScript/engine path cannot meet it. Check if the problem is misuse of audio buses or permission timing rather than a missing API.

Approve a native adapter only if it fixes the documented gap with bounded scope, adds no required network service, and preserves a complete Touch fallback. An Android-specific solution does not automatically solve iOS. Add the platform boundary now; add native implementation only when evidence justifies it.

3. Optional Kotlin contract

Use Godot's v2 Android plugin architecture and the version-matched Godot Android library [S10]. Extend GodotPlugin, annotate exposed methods with `@UsedByGodot`, declare plugin metadata, and package through the v2 export-plugin workflow. Method names are exact; do not assume camelCase/snake_case conversion [S10].

Proposed singleton name: `RoarballAndroid`. GDScript accesses it behind a PlatformAdapter after `Engine.has_singleton(...)`. Missing plugin means supported fallback, not a crash. If the plugin owns mic capture, expose permission/capture/error state and bounded RMS summaries; it should not move the ball or mutate UI nodes.

Suggested contract (design only; signatures must be finalized against the selected plugin template):

```
getCapabilities() -> Dictionary
requestMicrophonePermission(requestId: String) -> void
startLevelCapture(captureId: String) -> void
stopLevelCapture(captureId: String) -> void
openAppSettings() -> void
signals:
    permissionResult(requestId, granted, canAskAgain)
    captureStarted(captureId)
    levelWindow(captureId, rms, clipRatio, windowSeconds)
    captureStopped(captureId)
    platformError(requestId, code)
```

The Android adapter releases its recorder on stop, pause, destruction, and errors. Never hold both Godot and native microphone capture simultaneously. Bound buffering and make stop idempotent. Marshal callback data to the appropriate engine thread; do not pass raw PCM through a UI bridge at 60 fps. Do not claim native noise cancellation without device evidence.

No compiled Kotlin library, Gradle module, or native capture implementation is included in this planning package. Conditional task RB-048 covers it and is excluded from the default import set.

4. Permission behavior

Declare the microphone permission in the Android export configuration. Request it only after the player's pre-prompt choice and handle the asynchronous result. Godot's OS permission API documents that a true return can mean already granted, not a completed new dialog [S08]. Test the selected version; do not use obsolete short permission-name examples blindly.

Fresh denial, permanent denial, revocation in Settings, and revocation while paused have explicit UX. Touch must remain available. Avoid permission requests at boot. Verify the merged manifest contains only permissions required by the actual shipped features.

5. Toolchain and export

Pin Godot 4.7.2 and matching export templates [S01]. Use the selected version's Android export guide for Java/SDK/tooling; the stable guide checked for this document recommends OpenJDK 17 [S09]. Capture the exact SDK packages, build tools, NDK, Gradle and Android plugin versions in the repository once the release template is installed. Do not guess a dependency matrix from old snippets.

The engine's build requirements and the store's target-API policy are separate. Check the current store policy before each submission; this document does not assert that a particular API level is sufficient for September 2026 publishing. Choose final minimum device/API support after the physical-device gate.

Create a debug APK for device testing and a signed release artifact using the store-appropriate format. Use the final publisher-owned package identifier, not an assumed ShieldTech domain. Keep keystores/passwords in secure release storage and CI secrets, never this ZIP or Git.

6. Build sequence

1. Install the pinned editor and matching templates; record `godot --version`.
2. Import the project headlessly once, then run pure GDScript and integration tests.
3. Validate catalog/schema/scene markers and check missing assets.
4. Export debug Android build with the selected renderer and microphone permission.
5. Run the device gate: frame timing, microphone lifecycle, no-feedback audio, pause/resume, and repeated scene entry.
6. Freeze toolchain and content version; produce signed release build only after QA approval.
7. Install the release build, not only the debug build, and repeat smoke/permission/save tests.

Signing details are intentionally absent. Use a clean CI workspace with no production credentials on pull requests. Engine caches, generated build folders, export credentials, and local saves must not be committed.

7. Suggested CI jobs

Static/content validation: JSON schema, dependency graph, localization keys, missing file references, script parse checks. Domain tests: shot power, scoring, unlocks, save migrations. Integration tests: small headless scenes where applicable. Device performance/audio tests are separate; headless success cannot certify GPU/audio behavior.

Use a documented headless import/check/test command suited to the implemented project. The reference `.gd` files in this package are illustrative and have not been parsed by a Godot executable here. Do not copy a green Python validation badge into a claim that the Android game compiles.

8. Optional iOS track

Godot documents a standalone iOS export workflow through macOS and Xcode [S14]. Kotlin does not provide iOS integration. If native iOS capture is required, it needs its own Swift/Objective-C boundary and tests. iOS build/signing/permission work is post-MVP and not a reason to embed Godot in Flutter.

QA, Acceptance, and Release Evidence

1. Evidence policy

A requirement is complete only with reproducible evidence on the relevant runtime. Written architecture is not a tested feature. A scene screenshot is not a successful physics run. A synthetic amplitude test is not an Android microphone test. All numbers below are proposed acceptance thresholds, not completed measurements.

For each failure record build hash, engine/export-template version, device/OS, renderer, input route, level ID, steps, expected/actual outcome, and non-sensitive logs. Do not record player audio to reproduce a voice-input bug; use synthetic fixtures or consented observations without retained PCM.

2. Proposed device matrix

Select actual inventory before G0 closes. Minimum set: one intended minimum-tier Android phone, one mid-tier target phone, and one newer phone with a different GPU/driver family. Include at least two OS generations where the support policy requires them, a notched/gesture-navigation device, and a tablet/aspect-ratio test. Device names remain unfilled until measured; no compatibility is asserted in this delivery.

Test built-in microphone, wired audio where available, and Bluetooth as a compatibility case. Bluetooth voice play may fall back to Touch if latency/route behavior is inadequate; communicate this instead of claiming every headset works. Desktop emulation is useful but does not replace the mobile gate.

3. Acceptance test catalog

Test ID	Scenario	Expected result
QA-001	Fresh install, choose Touch.	No mic prompt; complete CC01.
QA-002	Fresh install, choose Voice and grant.	Calibration starts only after action/permission.
QA-003	Deny or permanently deny mic.	Full Touch route; no prompt loop.
QA-004	Revoke mic in OS settings.	Safe recovery; no stale Listening state.
QA-005	Stay silent during shot hold.	No accepted shot/stroke.
QA-006	Single short tap/clap noise.	Below duration gate; no accidental launch.
QA-007	Sustained qualified soft/strong input.	Distinct reachable powers, capped at 1.0.
QA-008	Opposite-phase stereo fixture.	Energy is not canceled by channel averaging.
QA-009	All-zero/nonfinite/clipped/overrun input.	Bounded error handling; no NaN physics.
QA-010	Irregular window/update durations.	Frame-time-aware smoothing behaves consistently.
QA-011	Release twice / two fingers.	At most one impulse and one stroke.
QA-012	Cancel gesture and >3 s hold.	No auto-fire; zero added strokes.
QA-013	Background during capture/warmup/commit pending.	Pending shot canceled; capture closed.

Test ID	Scenario	Expected result
QA-014	Background while rolling.	Simulation pauses; no new impulse on resume.
QA-015	Change audio route mid-hold.	Cancel and mark calibration stale.
QA-016	Thirty capture/start/stop cycles.	No duplicate owner, buffer leak, or lingering mic indicator.
QA-017	Game audio active during capture.	No voice monitoring; game does not power itself.
QA-018	Same normalized Touch/Voice input.	Same command/impulse; outcome within same-build tolerance.
QA-019	Positive/zero/out-of-range shot power.	Reject zero/invalid; clamp only valid range contract.
QA-020	Shoot while rolling or settling.	Action rejected; no hidden stroke.
QA-021	Flat roll distance sweep.	Stable stopping-distance bands at 25/50/75/100%.
QA-022	Ramp and high-speed thin obstacle.	No unacceptable tunneling or invisible snag.
QA-023	Slow ball on slope / midair apex.	No false safe rest.
QA-024	Cup slow entry, fast pass, high flyover, underside.	Only eligible entry completes once.
QA-025	Kill plane and kill volume fire together.	One penalty; one reset.
QA-026	Last safe anchor blocked.	Valid fallback anchor/spawn; no reset loop.
QA-027	Portal pair exit and re-entry.	Correct direction, clear exit, latch prevents loop.
QA-028	Pad holds overlapping ball.	One impulse per eligible entry.
QA-029	Moving gate pause/resume.	Phase preserved; no wall-clock teleport.
QA-030	Ball stuck for watchdog period.	Explicit recovery option, not silent teleport.
QA-031	Twelfth shot completes vs misses vs falls.	Win at <=12; failure otherwise; penalties correct.
QA-032	Par, par+1, par+2 finishes.	3/2/1 stars; failed result gives 0.
QA-033	Replay worse result; duplicate completion.	Best data retained; no extra unlock mutation.
QA-034	Complete CC06 with one star.	PP01 and Panda unlock.
QA-035	Complete PP06.	Robot unlock; final result navigates correctly.
QA-036	Truncated/invalid/future-version save.	Valid backup or clear recovery, no crash.
QA-037	Kill process during each save step.	Prior durable progress remains recoverable.
QA-038	Disk-write failure.	Save warning, retry, no false durable-success claim.
QA-039	Invalid catalog/scene marker/portal pair.	Build validation or safe loading error.
QA-040	All twelve holes independent playtest.	Successful reference routes and meaningful variation.
QA-041	Narrow/tall/notched/tablet UI.	No clipped controls; course/cup readable.
QA-042	Every screen pressed/disabled/error state.	Consistent feedback and no input fall-through.
QA-043	Reduced Motion / Touch-only full run.	Equal completion/rewards; no mandatory sound cues.
QA-044	Android Back through sheets and gameplay.	Correct stack; capture canceled safely.
QA-045	Thirty level enter/exit/retry cycles.	One session, no runaway nodes/resources.
QA-046	Fifteen-minute gameplay soak per quality tier.	Frame-time/memory/thermal targets met or documented blocker.

Test ID	Scenario	Expected result
QA-047	Storage/network/log inspection.	No retained/transmitted PCM; no hidden network SDK.
QA-048	Signed release install and offline smoke.	Same core behavior as tested debug build.

4. Unit and integration suites

Domain tests cover calibration math, quantile selection, smoothing, clamp/rejection, shot curve, star thresholds, attempt limits, sequential unlocks, and save merge/idempotency. Use injected clocks and synthetic level windows. Avoid game-state changes depending on wall clock.

Scene tests exercise the ball/controller boundary, start-stop capture tokens using a fake provider, cup eligibility, fall coalescing, portal transforms, pad latching, gate motion, safe reset, and loading cleanup. Assertions must distinguish simulation ticks from rendered frames.

Content validation checks schema and authored scene invariants. A metadata file saying “solvable” is not proof. Attach QA evidence to each final hole, including route screenshots and achieved strokes in the pinned build.

5. Performance protocol

Measure release-like builds after a warmup run. Use at least 15 minutes of repeated play per device/quality tier; track frame-time distribution, memory trend, loading hitches, and thermal slowdown. Proposed targets: 60 fps target tier and stable 30 fps low tier, with 60 Hz physics on both. If p95 frame time exceeds the chosen tier budget, identify CPU/GPU/UI/audio cost before cutting mechanics.

Initial budgets in PRD are adjustable only through an explicit decision with evidence. Test the most demanding actual level and worst-case particles. No performance result in this ZIP has been measured on a phone.

6. Usability protocol

Ask testers to select an input mode, take a gentle shot, take a strong shot, cancel, recover after a fall, use Overview, and switch to Touch. Observe without coaching until they get stuck. Record confusion around release-to-shoot versus power slider, whether the target is hidden by UI, and whether they feel pressured to be loud.

Afterward ask which moments felt intentional, which failures felt unfair, and whether voice adds enjoyment over Touch. Do not interpret “funny once” as durable retention. If voice control is frustrating, prioritize calibration/feedback rather than more rewards.

7. Definition of done

An implementation task needs code/assets, review, relevant automated tests, reproducible manual evidence where necessary, no new errors/warnings accepted without explanation, and updated documentation if contracts change. Visual tasks require device-scale captures, not only editor screenshots. Native tasks require physical-device evidence and fallback tests.

A release requires all P0 gates, no unresolved critical/major defects, content validation for twelve holes, first-run/offline/save recovery evidence, signed-build smoke, asset/license inventory, privacy-copy audit, and current store submission review. Store approval is not guaranteed by a technical checklist.

Delivery Plan and GitHub Workflow

1. Milestones without invented dates

Milestone	Outcome	Exit gate
M0 Foundation & feasibility	Toolchain, device inventory, renderer/physics/capture spikes.	Capture lifecycle and mobile rendering risks understood.
M1 Playable loop	One graybox hole, Touch and Voice, scoring/reset/results.	End-to-end physical-device run.
M2 Polished vertical slice	One production-quality course plus real menus/ HUD.	Approved visual/interaction evidence.
M3 MVP content & progression	Twelve holes, two worlds, three cosmetics, robust local saves.	Independent level validation and progression run.
M4 Hardening & Android release	Performance, accessibility, privacy, stability, signing.	Signed release candidate passes gate.
Later / conditional	Kotlin adapter only if justified; iOS/new content afterward.	Separate approved scope.

Do not set dates until actual team capacity and spike evidence exist. Task sizes are relative complexity (S/M/L), not time estimates or a commitment. P0 denotes a gate/critical-path item; P1 is needed for MVP but can follow the core path; P2 is deferred or conditional.

2. Critical path

Toolchain -> device/renderer/audio spike -> shot state and Touch physics -> microphone calibration and unified command -> complete graybox loop -> polished slice -> remaining content -> stability/privacy/performance -> signed release.

Art can explore mascot/module silhouettes during M0, but do not build twelve elaborate environments before the renderer and visual-slice gates pass. UI theme and save domain work can proceed in parallel after contracts are stable.

3. Task package

`tasks/backlog.json` is the canonical proposed backlog. `tasks/backlog.csv` is a convenience export, not a native GitHub import promise. Individual Markdown issues contain goals, implementation steps, acceptance checks, dependencies, evidence, and scope.

`tools/import_github_issues.py` uses GitHub CLI, which supports creating issues with titles and body files [S15]. It defaults to a local preview with no network/writes. `--apply --repo OWNER/REPO` is required to create actual issues. Default import includes only MVP tasks. Native/platform-later work requires `--include-conditional` or `--include-later` explicitly.

The importer uses stable issue IDs in titles/bodies and checks existing repository issue titles on apply to avoid ordinary duplicate reruns. It does not create a repository, set milestones, link GitHub Projects, or close existing issues. Milestone/priority/dependency metadata remains in each body. Dependencies use stable RB IDs; actual GitHub issue-number links can be added after import.

4. Working conventions

Small pull requests should reference RB IDs and explain the reproduced behavior, implementation, tests, visual/device evidence, and remaining risks. The included `.github` templates are optional repository setup material.

Keep a decision log for engine changes, voice thresholds, physical tuning, art budgets, and scope changes. Do not bury a Kotlin integration decision inside a feature branch. An AI agent must report its actual tests and failures; it must not describe untested screenshots as a finished game.

5. First implementation batch

Start RB-001 through RB-006 and the relevant shot-domain tasks. Produce a device report before expanding to expensive art. Then implement the full one-hole Touch loop, add Voice through the same command, and submit the first visual slice. Use the exact handoff prompt in the root file to keep the agent focused.

Decisions, Risks, and Open Questions

Architecture decisions

ADR-001: One Godot runtime owns both UI and world. Rationale: avoid a second rendering/input/lifecycle boundary for a standalone game. Revisit only if this game becomes part of a larger existing application.

ADR-002: Typed GDScript is the default gameplay language. Kotlin is a conditional Android adapter, never a parallel gameplay layer. No mandatory native addon is selected.

ADR-003: Integrated Jolt is the initial physics choice, pinned with the engine [S03]. Validate actual ball/ramp/portal behavior before content scaling; engine choice is not proof of appropriate golf tuning.

ADR-004: Offline local progression and cosmetic-only rewards. No account, economy, advertising, purchases, or analytics SDK in MVP. Reference art containing these elements is explicitly superseded.

ADR-005: Voice uses local calibrated amplitude; Touch is equivalent. No speech model or actual scream threshold. Hardware capture lifetime must be measured independently of UI state.

ADR-006: Twelve authored holes across two worlds; one polished vertical slice precedes full content. Par and power bands remain provisional until playtested.

ADR-007: Menus use Godot Controls and theme resources. Reference PNGs are review material only; they are not production UI layers or 3D assets.

Risk register

Risk	Impact	Mitigation / owner
Microphone remains active outside interaction.	Privacy failure; possible native work.	G0 physical indicator/lifecycle test; audio/platform owner.
Ambient noise or automatic gain makes power unpredictable.	Frustration; false shots.	Per-route calibration, valid-duration gate, clear Touch fallback; audio owner.
Game music powers the next shot.	Broken control.	Silent monitoring path and ducked output during capture; audio owner.
Concept art drives excessive geometry/effects.	Low frame rate or scope explosion.	One measured mobile slice and modest modular kit; art/tech lead.
Ball never settles or tunnels through edges.	Unplayable holes.	Rolling-resistance tuning, CCD tests, simplified collision; gameplay owner.
Portal/pad triggers repeat.	Infinite acceleration or loops.	Per-shot IDs, pair/contact latches, regression scenes; gameplay owner.
Physics changes invalidate authored pars.	Unfair progression.	Versioned tuning and solution revalidation; design/QA.
Save write interruption loses progress.	Trust loss.	Validate/backup/recover and kill-step tests; systems owner.
AI implementation delivers a UI facade.	False completion.	Real-device video, node/code inspection, strict task evidence; lead.

Risk	Impact	Mitigation / owner
Child/family positioning adds compliance work.	Release risk.	Publisher decision and current policy review before submission.
Working name conflicts with existing product.	Branding/release risk.	Commercial name/trademark review; publisher.

Open decisions with defaults

Final Android minimum device/API: default is unresolved until M0 measurement; do not advertise compatibility early. Default input choice: show Voice and Touch equally on first run; do not default into a permission dialog. Renderer: evaluate Mobile first, Compatibility fallback as an explicit tested target [S02].

Monetization: none in MVP. iOS: later. Kotlin: absent unless RB-048 is approved with reproduction evidence. Final art tools/font/license selection: team-owned approval; no font binaries or third-party art packs are bundled.

Change policy

Changes affecting controls, privacy copy, physics, stars, unlocks, or engine version require an updated decision entry, impacted-task list, and targeted regression plan. Adding new mockups does not automatically expand scope. Any business request for analytics, recording, accounts, or purchases is a separate privacy/security review.

Sources and Verification Boundaries

All sources were checked September 5, 2026. Primary official engine/API documentation and the GitHub CLI manual are used for factual platform capabilities. Stable documentation can move; implementation must compare it with the pinned engine version.

References such as [S06] resolve below. Gameplay thresholds, budgets, scope, UI dimensions, level designs, and milestones are proposed product decisions, not externally measured facts. No page is reproduced in full.

S01 — Godot official download archive

<https://godotengine.org/download/archive/>

Supports: Stable baseline: 4.7.2 listed with release date August 18, 2026. Version selection, not local binary verification.

S02 — Godot overview of renderers

<https://docs.godotengine.org/en/stable/tutorials/rendering/renderers.html>

Supports: Mobile and Compatibility roles, features, and hardware tradeoffs.

S03 — Godot using Jolt Physics

https://docs.godotengine.org/en/stable/tutorials/physics/using_jolt_physics.html

Supports: Integrated physics engine and relevant behavioral differences.

S04 — Godot RigidBody3D API

https://docs.godotengine.org/en/stable/classes/class_rigidbody3d.html

Supports: Impulse control, CCD, rigid-body properties; not application-specific tuning.

S05 — Godot AudioStreamMicrophone API

https://docs.godotengine.org/en/stable/classes/class_audiostreammicrophone.html

Supports: Microphone stream and enabled-input requirement.

S06 — Godot AudioEffectCapture API

https://docs.godotengine.org/en/stable/classes/class_audioeffectcapture.html

Supports: Stereo floating-point capture frames, ring buffer, availability/discard APIs.

S07 — Godot ProjectSettings API

https://docs.godotengine.org/en/stable/classes/class_projectsettings.html

Supports: Audio input permission caveats and mix/input rate distinctions.

S08 — Godot OS API

https://docs.godotengine.org/en/stable/classes/class_os.html

Supports: Permission request semantics and platform limits.

S09 — Godot exporting for Android

https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_android.html

Supports: Export setup and Java/SDK guide; not current store-policy certification.

S10 — Godot Android plugins

https://docs.godotengine.org/en/stable/tutorials/platform/android/android_plugin.html

Supports: v2 plugin architecture, Kotlin/Java methods, metadata and export packaging.

S11 — Godot available 3D formats

https://docs.godotengine.org/en/stable/tutorials/assets_pipeline/importing_3d_scenes/available_formats.html

Supports: Blender/gLTF import workflow.

S12 — Godot multiple resolutions

https://docs.godotengine.org/en/stable/tutorials/rendering/multiple_resolutions.html

Supports: Viewport/stretch design; our pixel sizes and layout are proposals.

S13 — Godot saving games

https://docs.godotengine.org/en/stable/tutorials/io/saving_games.html

Supports: FileAccess/JSON persistence primitives; transactional protocol is our design.

S14 — Godot exporting for iOS

https://docs.godotengine.org/en/stable/tutorials/export/exporting_for_ios.html

Supports: Separate macOS/Xcode export track, outside default MVP.

S15 — GitHub CLI issue create manual

https://cli.github.com/manual/gh_issue_create

Supports: Issue creation flags used by the optional local import helper.

Supplied visual provenance

Seven existing Roarball concept PNGs were recovered from the active conversation files. They are copied without new generation or editing into design/references. No new images were generated for this documentation task. They are not layered art, GLB models, production textures, or tested gameplay screenshots.

Not verified in this delivery

No Godot runtime, Android/iOS build, GPU benchmark, physical microphone capture, name clearance, store-policy compliance, Kotlin compilation, or level solvability has been verified. Package-level checks are reported separately and must not be confused with runtime QA.

Task overview

60 proposed tasks: **56 MVP**, one conditional Kotlin task, and three later-scope tasks. Individual issue bodies in tasks/issues contain implementation steps, acceptance criteria, dependencies, and required evidence. No remote issues have been created.

Milestones are evidence gates, not calendar commitments. IDs remain stable and are not necessarily dependency order. The import helper orders selected issues by dependencies and previews locally by default.

ID	Task	Gate	Priority
RB-001	Pin Godot toolchain and initialize repository	M0	P0
RB-002	Select physical Android test matrix	M0	P0
RB-003	Benchmark Mobile and Compatibility renderers	M0	P0
RB-004	Validate Jolt ball, ramp, and collision behavior	M0	P0
RB-005	Prove mic permissions and hardware release lifecycle	M0	P0
RB-006	Approve MVP flows and reconcile concept mockups	M0	P0
RB-007	Implement typed shot, score, and tuning domain contracts	M1	P0
RB-008	Implement authoritative gameplay state machine	M1	P0
RB-009	Create level wrapper and scene-marker contract	M1	P0
RB-010	Implement ball impulse and rolling resistance	M1	P0
RB-011	Implement playfield aiming and input ownership	M1	P0
RB-012	Implement complete Touch shot controls	M1	P0
RB-013	Implement Ready, Follow, and Overview cameras	M1	P1
RB-014	Implement support, rest detection, and safe anchors	M1	P0
RB-015	Implement out-of-bounds and stuck recovery	M1	P0
RB-016	Implement cup capture and completion ownership	M1	P0
RB-017	Implement stars, attempt limits, and unlock rules	M1	P0
RB-018	Build shared Godot UI theme and safe-area shell	M1	P1
RB-019	Implement mode choice and contextual permission UX	M1	P0
RB-020	Implement bounded VoiceInputService and silent monitoring	M1	P0
RB-021	Implement room/soft/strong calibration	M1	P0
RB-022	Implement preview power and stable release selection	M1	P0
RB-023	Connect hold/release/cancel voice gestures	M1	P0
RB-024	Handle pause, background, route changes, and scene disposal	M1	P0
RB-025	Implement game audio ducking and nonverbal feedback	M1	P1
RB-026	Pass one-hole end-to-end playable-loop gate	M1	P0
RB-027	Create lion mascot production asset and expressions	M2	P1
RB-028	Create modular course kit and collision wrappers	M2	P1
RB-029	Establish mobile lighting, materials, and quality tiers	M2	P1
RB-030	Implement Home and loading/error navigation	M2	P1

ID	Task	Gate	Priority
RB-031	Implement two-world twelve-hole selection	M2	P1
RB-032	Polish calibration UI and recovery states	M2	P1
RB-033	Polish Ready/Capturing/Rolling gameplay HUD	M2	P1
RB-034	Implement Pause, Settings, and Android Back behavior	M2	P1
RB-035	Implement truthful results and save-warning UI	M2	P1
RB-036	Approve one polished visual and interaction slice	M2	P0
RB-037	Implement moving gate obstacle	M3	P1
RB-038	Implement paired portals and exit validation	M3	P1
RB-039	Implement one-shot bounce pad	M3	P1
RB-040	Author and validate Cloud Cliffs holes 1–3	M3	P1
RB-041	Author and validate Cloud Cliffs holes 4–6	M3	P1
RB-042	Author and validate Portal Peaks holes 1–3	M3	P1
RB-043	Author and validate Portal Peaks holes 4–6	M3	P1
RB-044	Implement three cosmetic balls and collection screen	M3	P1
RB-045	Implement validated local progress/settings and recovery	M1	P0
RB-046	Implement content catalog and scene validators	M1	P1
RB-047	Run all-hole playtest and balance review	M3	P0
RB-048	Conditional: add narrowly scoped Kotlin input/platform adapter	Conditional	P2
RB-049	Externalize strings and prepare English/Indonesian localization	M3	P1
RB-050	Verify accessibility, reduced motion, and equal Touch path	M4	P0
RB-051	Build automated domain/state/scene regression suite	M4	P0
RB-052	Profile and optimize sustained mobile performance	M4	P0
RB-053	Audit privacy, permissions, storage, and dependency provenance	M4	P0
RB-054	Run mobile interruption and navigation stability suite	M4	P0
RB-055	Create reproducible signed Android release candidate	M4	P0
RB-056	Prepare store materials, policy review, and asset/name clearance	M4	P1
RB-057	Approve MVP release gate and hand over operations	M4	P0
RB-058	Later: evaluate standalone iOS export and audio lifecycle	Later	P2
RB-059	Later: evaluate wind/lava worlds and daily challenges	Later	P2
RB-060	Later: review online analytics or monetization as separate scope	Later	P2

Original gameplay

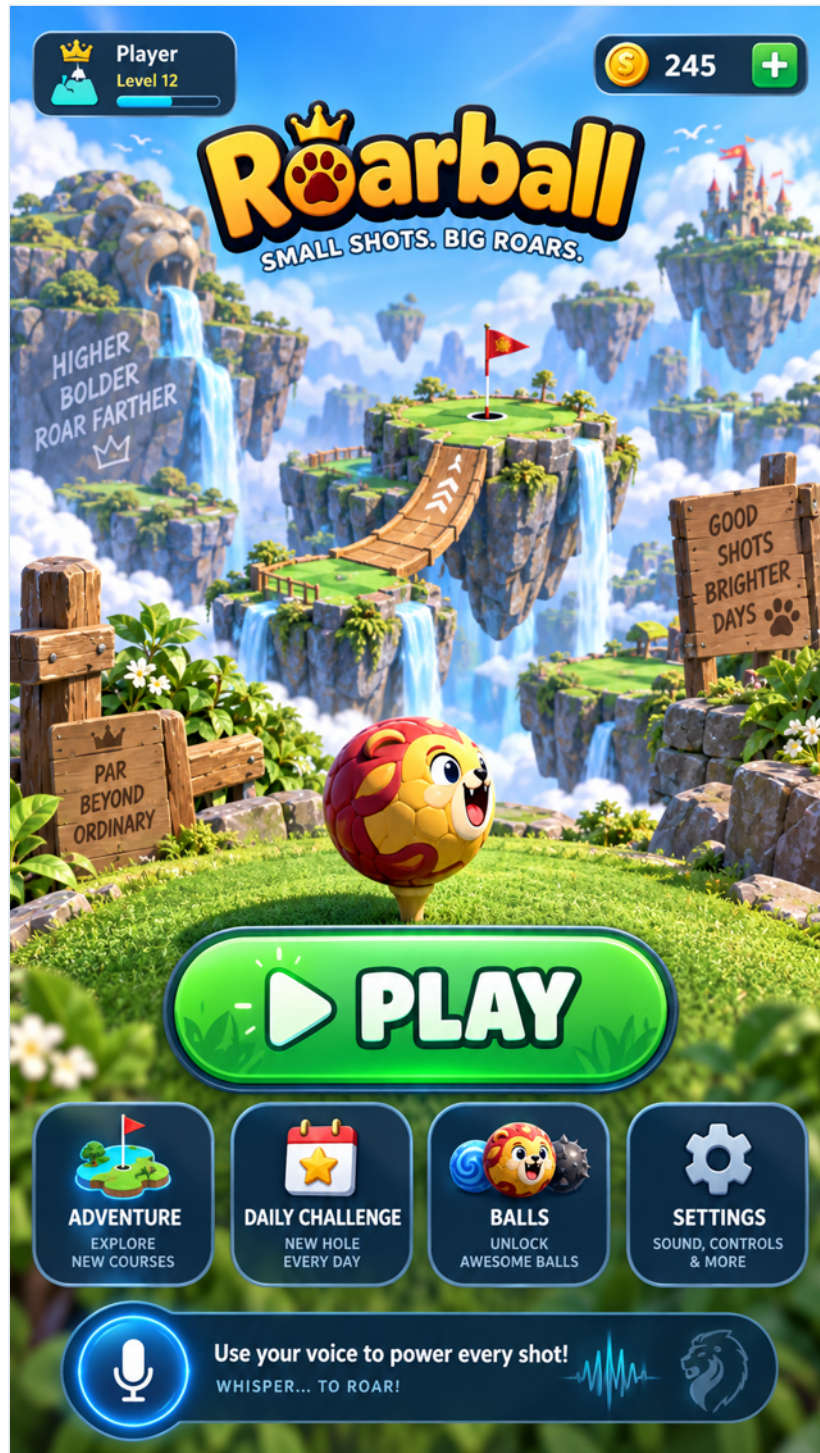
MVP interpretation: Keep the lion, floating turf, flags and playful depth. Use a compact header, real shot/cancel states and no currency UI.



Existing concept image, not a real-time engine screenshot. Original full-resolution PNG is included. Build the interface as Godot Controls and the world as real 3D scenes.

Home

MVP interpretation: Keep the hero and clear Play button. MVP navigation is Play, Map, Balls and Settings. Remove daily challenge, account level and coins.



Existing concept image, not a real-time engine screenshot. Original full-resolution PNG is included. Build the interface as Godot Controls and the world as real 3D scenes.

World map

MVP interpretation: MVP has Cloud Cliffs and Portal Peaks, six holes each. Remove hearts, shop and extra worlds. Completion unlocks the next hole.



Existing concept image, not a real-time engine screenshot. Original full-resolution PNG is included. Build the interface as Godot Controls and the world as real 3D scenes.

Voice calibration

MVP interpretation: Implement explicit Room / Soft / Strong stages, a Touch alternative, real permission/error states and verified capture lifetime.



Existing concept image, not a real-time engine screenshot. Original full-resolution PNG is included. Build the interface as Godot Controls and the world as real 3D scenes.

Advanced course

MVP interpretation: Use reusable ramps, gates, one portal pair and pads. Wind is deferred. Concept composition is not proof a layout is solvable.



Existing concept image, not a real-time engine screenshot. Original full-resolution PNG is included. Build the interface as Godot Controls and the world as real 3D scenes.

Ball collection

MVP interpretation: Lion, Panda and Robot only, with identical physics. No rarity, shop, currency, trails or emote economy in MVP.



Existing concept image, not a real-time engine screenshot. Original full-resolution PNG is included. Build the interface as Godot Controls and the world as real 3D scenes.

Results

MVP interpretation: Show earned stars, strokes, par and world progress. Remove fabricated whisper/accuracy percentages, gems and required sharing.



Existing concept image, not a real-time engine screenshot. Original full-resolution PNG is included. Build the interface as Godot Controls and the world as real 3D scenes.